

RESEARCH

Open Access



Improved integral neural distinguisher model for lightweight cipher PRESENT

Wanqing Wu^{1,2} and Mingyu Guo^{1,2*}

Abstract

PRESENT is a lightweight block cipher, which has attracted many scholars to research its security. In 2022, Zahednejad et al. proposed the integral neural distinguisher on round-reduced PRESENT. In this paper, a new integral neural distinguisher for PRESENT is constructed. In contrast to Zahednejad's works, the proposed integral neural distinguisher can improve the number of attack rounds in one round. This paper proposes a new data format $(invP_0^n, invP_1^n, \dots, invP_{15}^n, invS_0^n, invS_1^n, \dots, invS_{15}^n)$, which can expose more features of PRESENT previous round ciphertext. Simultaneously, this paper incorporates MBConv module into the convolutional layers of DenseNet, which enable the neural network to identify a greater variety of features in ciphertext. The data format of the paper is analysed. The results of the analysis show that the data format in this paper is able to identify more features compared to Zahednejad's data format. Further to this, experiments are performed on PRESENT using the integral neural distinguisher in this paper. The experimental results show that the changes make to the neural network and data format have improved the accuracy of distinguishers. Finally, key recovery attacks are conducted on the integral neural distinguishers of SmallPresent-(8) to demonstrate the efficacy of the distinguisher proposed in this paper. The results demonstrate that the key recovery success rates for 5-round and 6-round are 98% and 90%, considering error bits within a range of two bits.

Keywords Integral Attack, Neural Network, Key Recovery, SPN, PRESENT

Introduction

With the widely application of devices such as sensors and RFID, the Internet of Things (IoT) is increasingly becoming a part of daily life. However, the pursuit of enhanced productivity and convenience leads to the compression of storage and computing resources, which makes traditional ciphers (e.g. AES) ineffective in ensuring the data security and privacy of IoT devices. In order to ensure data security and privacy in resource-constrained environments, PRESENT was proposed by Bogdanov (2007), which is designed to have low power

consumption, low throughput and high efficiency. Since PRESENT was proposed, several cryptanalysis methods such as differential analysis, linear attacks, integral analysis, algebraic analysis, bypass cube analysis, have been employed to scrutinize its security (Bogdanov 2007; Wu and Wang 2013; Xiang et al. 2016; Z'aba et al. 2008; Jain et al. 2020; Nakahara et al. 2009; Yang et al. 2009; Cho 2010; Wang 2008; Collard and Standaert 2009).

Integral attack (Knudsen and Wagner 2002) was proposed by Knudsen et al., which borrows from square attack (Daemen et al. 1997), saturation attack (Lucks 2001) and multiset attack (Biryukov and Shamir 2001). Initially, the integral attack was mainly applied to the byte-based ciphers. However, at FSE 2008, bit-based integral attack was proposed and applied to analyse Noekeon, Serpent and PRESENT by Z'aba et al. (2008). This improvement solve the problem that integral cryptanalysis cannot be applied to bit-based ciphers. In

*Correspondence:

Mingyu Guo
18617535304@163.com

¹ School of Cyber Security and Computer, Hebei University, Baoding 071002, Hebei Province, China

² Key Laboratory on High Trusted Information System in Hebei Province, Baoding 071002, Hebei Province, China

2013, Wu and Wang (2013) used the properties of PRESENT's Sbox to evaluate algebraic degrees and found a 7-round integral distinguisher. In 2016, both Xiang et al. (2016) and Todo and Morii (2016) found a 9-round integral distinguisher by bit-based division property (Todo 2015), and 12-round PRESENT-80 and 13-round PRESENT-128 were attacked by Todo and Morii (2016) via using the 9-round integral distinguisher. In 2018, Wang et al. (2018) proposed an algebraic method and used it to attack 10-round PRESENT-80 and 12-round PRESENT-80. At the same time, they proposed a key partition technique to attack 11-round PRESENT-80 and 13-round PRESENT-80.

In recent years, the widely application of deep learning has provided many new approaches to cryptanalysis. At CRYPTO 2019, the great potential of machine learning in cipher identification was demonstrated for the first time by Ghor (2019). Ghor designed a differential neural distinguisher for the SPECK using ResNet (He et al. 2016) and performed key recovery experiments. This result has led many researchers to investigate neural network assisted cryptanalysis. In 2021, Chen and Yu (2021) improved Ghor's neural distinguisher using multiple ciphertext pairs as the neural network input. They improved the neural network accuracy using different output differences. Inspired by Ghor's work, in 2020, Jain et al. (2020) developed a differential neural distinguisher and applied it to round-reduced PRESENT, which identified 3–5 rounds of PRESENT data from random data with a high probability. Hou et al. (2020) proposed a combination of neural networks and linear cryptanalysis to identify 3–5 rounds of DES, which allow the neural networks gaining the ability to distinguish the linear expressions between the DES and random data. In 2022, Zahednejad and Lyu (2022) firstly proposed combining integral attacks with neural networks. They proved that neural network can capture the integral features of ciphertexts, and their integral neural distinguisher can improve the number of identify rounds by 1–2 rounds compared to the classical integral distinguisher in a variety of cipher structures (e.g. SPN, Feistel, ARX, etc.). In the case of PRESENT, they achieve a 7-round integral neural distinguisher using a 5-round classic integral distinguisher. But as the number of rounds increases, the accuracy of the neural distinguisher decreases from 99% in 5 rounds to 58.1% in 7 rounds. Notably, the 8-round PRESENT integral neural distinguisher transitioned into a process of random guessing. In order to increase the number of rounds identified by the neural network, this paper designs a new data preprocessing method based

on the characteristics of the PRESENT and the integral attack.

Our contributions

In this paper, according to the characteristics of PRESENT and integral attack, a new data preprocessing method is adopted. It not only ignores the effects of permutation and subkey addition, but also identifies more features of the previous round ciphertexts. Simultaneously, this paper improves the number of distinguish rounds to 8 rounds with accuracy of 57.32% via using multi-multiset data format, and the convolutional layer of DenseNet is modified by using the MBConv module, which reduces the training time successfully and improves the accuracy of the neural network. The accuracy for rounds 5–7 are 99.58%, 99.13% and 82.03%, respectively. Finally, key recovery attacks are performed on the integral neural distinguishers of SmallPresent-(8). The results demonstrate that the key recovery success rates of the 5-round and 6-round neural distinguishers are 98% and 90% respectively, considering error bits within a range of two bits.

Preliminaries

Brief description of PRESENT and SmallPresent-(size)

PRESENT

PRESENT (Bogdanov 2007) is an SPN block cipher for the resource-constrained environments. It is a 31-round ultra-light block cipher with a block size of 64 bits. The underlying structure of PRESENT has three layers in every round: AddRoundKey layer, SBoxLayer and PLayer. In the AddRoundKey layer, the current state is XORed to the round key. Then, one 4-bit Sbox is applied by the SBoxLayer repeatedly. Finally, all bits are shifted in the PLayer. At the end of the last round, the current state is XORed to the round key to get the ciphertext. The encryption flow of the whole algorithm is shown in Fig. 1.

According to Fig. 1, if we take the result of the plaintext passing through the addRoundKey layer as the initial encryption state. Then, the order of the three layers can be changed from AddRoundKey layer, SBoxLayer, PLayer to SBoxLayer, PLayer, AddRoundKey layer. Let Y and Z denote the n_{th} round input and output of the initial encryption state. Then the encryption and decryption process can be expressed in the equations:

$$Z = P(S(Y)) \oplus k_n. \quad (1)$$

$$Y = S^{-1}(P^{-1}(Z \oplus k_n)). \quad (2)$$

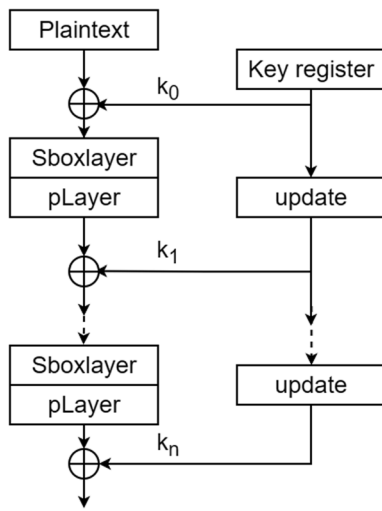


Fig. 1 PRESENT encryption flowchart

Where P , S denote the permutation and substitution in the encryption process. P^{-1} and S^{-1} denote the inverse-permutation and inverse-substitution in the decryption process. And k_n denote the subkey of the n_{th} round.

Key schedule This paper uses PRESENT with a key of 80 bits. The key is represented as $k_{79}k_{78}k_{77} \dots k_0$ and stored in the key register. The subkey for each round is consisted of the leftmost 64 bits of the key register. After each round of encryption, $k_{79}k_{78}k_{77} \dots k_0$ in the key register is updated as follows:

Step 1. $[k_{79}k_{78} \dots k_1k_0] = [k_{18}k_{17} \dots k_{20}k_{19}]$.

Step 2. $[k_{79}k_{78}k_{77}k_{76}] = S[k_{79}k_{78}k_{77}k_{76}]$.

Step 3. $[k_{19} \dots k_{15}] = [k_{19} \dots k_{15}] \oplus \text{round_counter}$.

Where *round_counter* represents the current encryption round.

Step 1: The key register is rotated by 61 bit positions to the left.

Step 2: The leftmost 4 bits of the key register are passed through the S-box.

Step 3: The value of *round_counter* is XORed to the $k_{19}k_{18}k_{17}k_{16}k_{15}$.

SmallPresent-(size)

In order to clearly describe the time complexity of attacking the PRESENT(Cho 2010; Collard and Stan-daert 2009), Leander (2010) designed its general variant SmallPresent-(size) ($0 < \text{size} \leq 16$, indicates the size of the PRESENT variant used by the user) based on PRESENT-80. SmallPresent-(size) represents a PRESENT

variant with a block size of $4 \times \text{size}$. The overall encryption process of SmallPresent-(size) is the same as that of PRESENT. The differences are as follows:

Player The permutation utilized in SmallPresent-(size) is defined by the following function, which shifts the bit state of *state* to the bit position $P(\text{state})$.

$$P(\text{state}) = \begin{cases} \text{size} \times \text{state} \bmod (4 \times \text{size} - 1), & \text{for } 0 \leq \text{state} < 4 \times \text{size} - 1 \\ 4 \times \text{size} - 1, & \text{for } \text{state} = 4 \times \text{size} - 1. \end{cases}$$

Inverse permutation can be described as:

$$P^{-1}(\text{state}) = \begin{cases} 4 \times \text{state} \bmod (4 \times \text{size} - 1), & \text{for } 0 \leq \text{state} < 4 \times \text{size} - 1 \\ 4 \times \text{size} - 1, & \text{for } \text{state} = 4 \times \text{size} - 1. \end{cases}$$

Key schedule SmallPresent's key scheduling is similar to that of PRESENT-80, with the exception that the subkey consists of the $4 \times \text{size}$ rightmost bits of PRESENT-80's subkey.

Classical integral distinguisher

Integral analysis was proposed by Daemen et al. when analysing the security of SQUARE (Daemen et al. 1997). Subsequently, Knudsen et al. formalized it based on the Square attack (Daemen et al. 1997), Saturation attack (Lucks 2001) and Multiset attack (Biryukov and Shamir 2001). It is a chosen-plaintext attack in which the adversary selects 2^N plaintexts (known as multiset) for a block cipher of size H bits. These plaintexts are binary strings consisting of active bits with different $N(N \leq H)$ bit arrangements and constant bits with same $H - N$ bit arrangements. Then the plaintexts are encrypted to obtain the corresponding ciphertexts. If the corresponding ciphertexts contains certain bits, whose XOR is always equal to 0, these bits are called balance bits. Further it claims that the integral distinguisher with 2^N selected plaintexts (or a data complexity of 2^N) is found. The XOR of the balance bits is the integral feature on which the integral attack relies.

Taking the PRESENT as an example, Xiang et al. (2016) proposed a 5-round integral distinguisher in which the rightmost 4 bits of the ciphertext are balance bits. The distinguisher fixes the rightmost 4 bits of the plaintext as active bits and the leftmost 60 bits of the plaintext as constant bits. The formats of plaintext and its associated ciphertext is shown in Fig. 2.

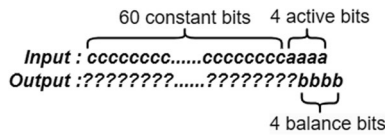


Fig. 2 PRESENT's 5-round classic integral distinguisher

Encrypt the selected plaintexts for 5 rounds to obtain a multiset containing 16 ciphertexts. Then, apply XOR to the ciphertexts:

$$C_0^5 \oplus C_1^5 \oplus C_2^5 \oplus C_3^5 \oplus \dots \oplus C_{15}^5. \quad (3)$$

The XOR of the balance bits in Eq. 3 is equal to 0. The adversary can decrypt the ciphertexts with their guess of the last-round key. If the XOR of the balance bits from the ciphertexts is equal to 0, the adversary has found the correct subkey.

DenseNet

DenseNet (Huang et al. 2017) was specially developed to improve accuracy caused by the vanishing gradient in neural networks. Unlike other networks (such as ResNet He et al. 2016 and ResNeXt Xie et al. 2017) establish short-circuited connections among layers. DenseNet introduces the concept of dense connectivity, which directly connects all layers in the network.

Assume that the output equation of the neural network is:

$$y_l = F_l(y_{l-1}).$$

Where y_l represents the mapping output of the F_l to the previous layer y_{l-1} . F_l usually includes BN, ReLU, Pooling, Conv and other operations.

For efficient communication among layers, DenseNet uses cross-channel connections to merge the outputs of previous layers into the current layer's mapping. The equation for each layer of DenseNet is as follows.

$$y_l = F_l([y_0, y_1, y_2, \dots, y_{l-1}]).$$

Where y_l denotes the mapping output of the F_l to all previous layers. The network structure of DenseNet is shown in Fig. 3.

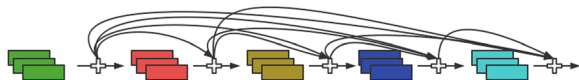


Fig. 3 DenseNet network structure

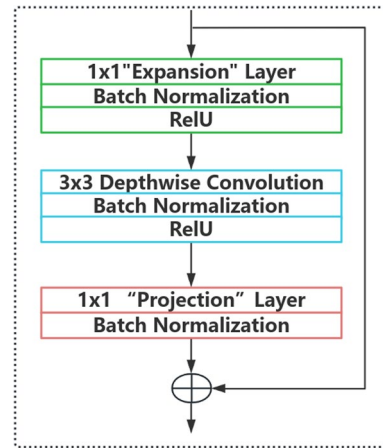


Fig. 4 MBConv module

MBConv

MBConv module is an inverted linear bottleneck layer with deeply separable convolution, which is commonly used for lightweight network structures (e.g., Efficientnet Tan and Le 2019, MobileNetV3 Howard et al. 2019, Mnasnet Tan et al. 2019). MBConv uses a lightweight depthwise separable convolution (Chollet 2017) to divide the standard convolution into channel-by-channel convolution and point-by-point convolution, thereby reducing the convolutional parameters while maintaining the same input parameters.

Figure 4 shows the main operations of MBConv: Firstly, a 1×1 kernel-size convolution is employed to increase the number of input channels. Secondly, a 3×3 kernel-size depth-separated convolution is employed to conduct channel-by-channel and point-by-point convolutional operations. Subsequently, a 1×1 kernel-size convolution is utilized to adjust the number of channels to the specified filter size. Finally, the inputs and outputs are added together.

Build 8-round integral neural distinguisher

Data format

Encrypting the plaintexts from Xiang et al. (2016)'s 5-round integral distinguisher for n ($5 \leq n \leq 8$) rounds:

$$C_0^n, C_1^n, C_2^n, C_3^n, \dots, C_{12}^n, C_{13}^n, C_{14}^n, C_{15}^n. \quad (4)$$

Where C_j^n denotes the result of encrypt the j_{th} plaintext for n rounds.

C_j^n is decrypted using Eq. 2, where the k_n is C_0^n . The following result is:

$$\begin{aligned}
& S^{-1}(P^{-1}(C_0^n \oplus C_0^n)), S^{-1}(P^{-1}(C_1^n \oplus C_0^n)), \\
& \dots \\
& S^{-1}(P^{-1}(C_{14}^n \oplus C_0^n)), S^{-1}(P^{-1}(C_{15}^n \oplus C_0^n)).
\end{aligned}$$

Based on Eq. 2, split the decryption process of C_j^n into inverse permutation and inverse substitution:

$$\begin{cases} \text{inv}P_j^n = P^{-1}(C_j^n \oplus C_0^n) \\ \text{inv}S_j^n = S^{-1}(\text{inv}P_j^n). \end{cases} \quad (5)$$

Where $\text{inv}P_j^n$ denotes the inverse permutation result after applying XOR to C_j^n and C_0^n . $\text{inv}S_j^n$ represents the inverse of S-Box.

Finally, the data format of this paper is:

$$\text{inv}P_0^n, \text{inv}P_1^n, \dots, \text{inv}P_{15}^n, \text{inv}S_0^n, \text{inv}S_1^n, \dots, \text{inv}S_{15}^n. \quad (6)$$

Figure 5 shows the data generation process. The specific data format generation process is as follows:

Firstly, we generate the set M randomly, which includes r multisets conforming to the 5-round integral distinguisher.

$$\begin{aligned}
& (M_{0_0}, M_{0_1}, M_{0_2}, \dots, M_{0_{13}}, M_{0_{14}}, M_{0_{15}}), \\
& (M_{1_0}, M_{1_1}, M_{1_2}, \dots, M_{1_{13}}, M_{1_{14}}, M_{1_{15}}), \\
& \dots \\
& (M_{r_0}, M_{r_1}, M_{r_2}, \dots, M_{r_{13}}, M_{r_{14}}, M_{r_{15}}).
\end{aligned}$$

Where M_{r_i} denotes the i_{th} plaintext in the r_{th} multiset.

Secondly, The plaintext set M is encrypted for n rounds. We generate the master key K and binary sample label Y randomly. When $Y = 1$, the M is encrypted with K . When $Y = 0$, All plaintexts in the set M are replaced with unrestricted plaintexts. K is used to encrypt the replaced set M .

$$\begin{aligned}
& (C_{0_0}^n, C_{0_1}^n, C_{0_2}^n, \dots, C_{0_{13}}^n, C_{0_{14}}^n, C_{0_{15}}^n), \\
& (C_{1_0}^n, C_{1_1}^n, C_{1_2}^n, \dots, C_{1_{13}}^n, C_{1_{14}}^n, C_{1_{15}}^n), \\
& \dots \\
& (C_{r_0}^n, C_{r_1}^n, C_{r_2}^n, \dots, C_{r_{13}}^n, C_{r_{14}}^n, C_{r_{15}}^n).
\end{aligned}$$

Where $C_{r_i}^n$ denotes the result of encrypt the M_{r_i} for n rounds.

Finally, Eq. 6 converts the ciphertexts into the data format required by the neural network:

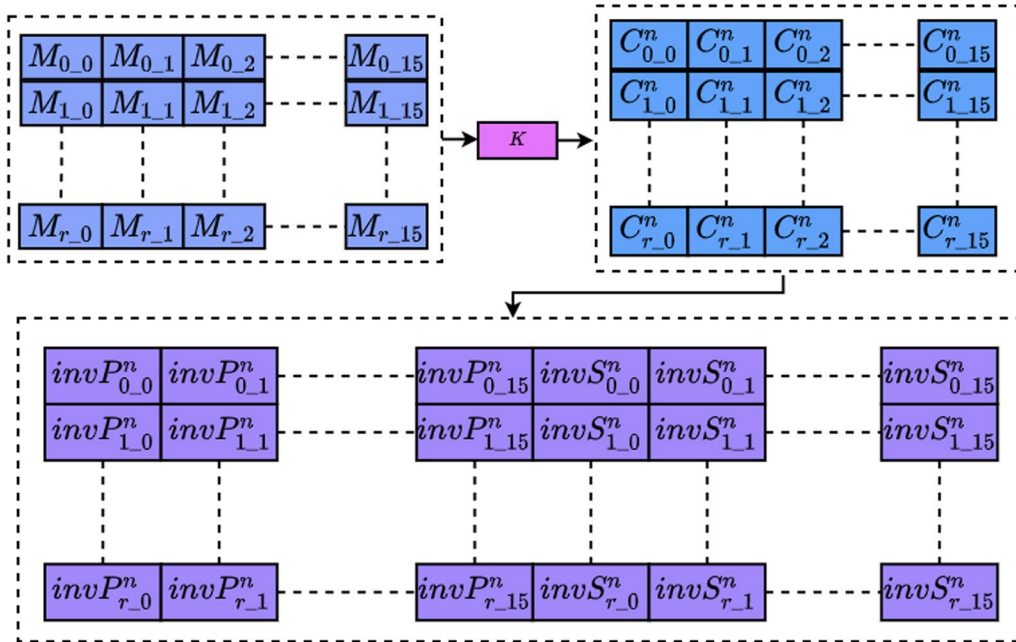


Fig. 5 Data generation process

$$\begin{aligned}
& invP_{0,0}^n, invP_{0,1}^n, \dots, invP_{0,15}^n, \\
& invS_{0,0}^n, invS_{0,1}^n, \dots, invS_{0,15}^n, \\
& invP_{1,0}^n, invP_{1,1}^n, \dots, invP_{1,15}^n, \\
& invS_{1,0}^n, invS_{1,1}^n, \dots, invS_{1,15}^n, \\
& \dots \\
& invP_{r,0}^n, invP_{r,1}^n, \dots, invP_{r,15}^n, \\
& invS_{r,0}^n, invS_{r,1}^n, \dots, invS_{r,15}^n.
\end{aligned}$$

Where $invP_{r,i}^n$ denotes the inverse permutation result after applying XOR to $C_{r,i}^n$ and $C_{r,0}^n$. $invS_{r,i}^n$ denotes the inverse S-Box result for $invP_{r,i}^n$.

Analysis of the data format

Zahednejad et al. used Eq. 4 as the input data format for the neural distinguisher. We propose a data format in “Data format” section based on Zahednejad’s data format.

Here, we compare the data format proposed in “Data format” section with Zahednejad’s data format. After analyzing the data format proposed in this paper, as well as the S-Boxes and the ciphertexts of the previous round, we find some underlying relations between them. These relations enable our method to improve the number of rounds and its accuracy compared to the work of Zahednejad and Lyu (2022). The reasons are as follows.

Proposition 1 5-round integral features can be obtained by $invP_0^5, invP_1^5, \dots, invP_{15}^5$ in Eq. 6.

Proof The $invP_0^5, invP_1^5, \dots, invP_{15}^5$ in round 5 is denoted by the ciphertexts:

$$\begin{aligned}
& P^{-1}(P(S(C_0^4)) \oplus C_0^5 \oplus P(S(C_0^4)) \oplus C_0^5) \\
& P^{-1}(P(S(C_0^4)) \oplus C_0^5 \oplus P(S(C_1^4)) \oplus C_0^5) \\
& \dots \\
& P^{-1}(P(S(C_0^4)) \oplus C_0^5 \oplus P(S(C_{15}^4)) \oplus C_0^5).
\end{aligned}$$

P^{-1} , P and C_0^5 are eliminated to obtain:

$$S(C_0^4) \oplus S(C_0^4), S(C_0^4) \oplus S(C_1^4), \dots, S(C_0^4) \oplus S(C_{15}^4). \quad (7)$$

Apply XOR to the above results:

$$S(C_0^4) \oplus S(C_1^4) \oplus S(C_2^4) \oplus S(C_3^4) \oplus \dots \oplus S(C_{15}^4).$$

The value of the bit is not changed by the operations of P or P^{-1} , so P^{-1} does not affect the extraction of integral features:

$$\begin{aligned}
& P^{-1}(C_0^5 \oplus C_1^5 \oplus \dots \oplus C_{15}^5) \\
& = P^{-1}(P(S(C_0^4)) \oplus k_5 \oplus \dots \oplus P(S(C_{15}^4)) \oplus k_5) \\
& = S(C_0^4) \oplus S(C_1^4) \oplus S(C_2^4) \oplus S(C_3^4) \oplus \dots \oplus S(C_{15}^4).
\end{aligned}$$

It follows that the 5-round PRESENT integral features extracted by $invP_0^5, invP_1^5, \dots, invP_{15}^5$. $\square$

The 5-round PRESENT integral distinguisher is improved to 7 rounds by Zahednejad using a neural network. In order to improve the number of attack rounds, we use Eq. 6 as a new data format. Proposition 1 shows that in the case of 5 rounds, the features extracted by $invP_0^5, invP_1^5, \dots, invP_{15}^5$ in Eq. 6 are equivalent to $C_0^5, C_1^5, C_2^5, \dots, C_{15}^5$ of Zahednejad. However, the number of identification rounds can be improved to 8 rounds by using Eq. 6 as a new data format, the reasons are given in Proposition 2 and Proposition 3.

Definition 1 Denote S_{m_inv} as the S^{-1} of $S(A) \oplus S(B)$, where $S(A)$, $S(B)$ denote two S-Boxes. S_{m_inv} has a form as:

$$S_{m_inv} = S^{-1}(S(A) \oplus S(B)). \quad (8)$$

Proposition 2 The XOR of the two S-Boxes inputs is acquired by S_{m_inv} with a non-negligible probability greater than 1/2.

Proof Denote x, y as the input value and output value of the i_{th} bit in the S-Box, respectively. The algebraic normal form (ANF) of PRESENT inverse S-Box:

$$\begin{cases}
x_3 = y_0 \oplus y_1 \oplus y_2 \oplus y_3 \oplus y_0 y_1 \oplus y_0 y_1 y_2 \oplus y_0 y_2 y_3 \\
x_2 = 1 \oplus y_3 \oplus y_0 y_1 \oplus y_0 y_2 \oplus y_0 y_3 \oplus y_1 y_2 \oplus y_1 y_3 \\
\quad \oplus y_0 y_1 y_2 \oplus y_0 y_1 y_3 \oplus y_0 y_2 y_3 \\
x_1 = y_0 \oplus y_1 \oplus y_3 \oplus y_0 y_2 \oplus y_1 y_3 \oplus y_2 y_3 \\
\quad \oplus y_0 y_1 y_2 \oplus y_0 y_1 y_3 \oplus y_0 y_2 y_3 \\
x_0 = 1 \oplus y_0 \oplus y_3 \oplus y_1 y_3.
\end{cases} \quad (9)$$

Assuming that there are two four-bit S-Boxes, $A = [A_0, A_1, A_2, A_3]$, $B = [B_0, B_1, B_2, B_3]$ as inputs to the two S-Boxes, $a = [a_0, a_1, a_2, a_3]$ and $b = [b_0, b_1, b_2, b_3]$ as the corresponding output to the two S-Boxes. Equation 9 is used to calculate $S_{m_inv} = S^{-1}(S(A) \oplus S(B))$. Where $S_{m_inv}^i$ represents the value of the i_{th} bit of S_{m_inv} .

$s_{m_inv}^0$ is derived as follows:

$$\begin{aligned}
 s_{m_inv}^0 &= 1 \oplus (a_0 \oplus b_0) \oplus (a_3 \oplus b_3) \oplus (a_1 \oplus b_1)(a_3 \oplus b_3) \\
 &= 1 \oplus a_0 \oplus b_0 \oplus a_3 \oplus b_3 \oplus a_1 a_3 \oplus a_1 b_3 \oplus b_1 a_3 \\
 &\quad \oplus b_1 b_3 \\
 &= 1 \oplus a_0 \oplus a_3 \oplus a_1 a_3 \oplus 1 \oplus b_0 \oplus b_3 \oplus b_1 b_3 \\
 &\quad \oplus 1 \oplus a_1 b_3 \oplus b_1 a_3 \\
 &= A_0 \oplus B_0 \oplus 1 \oplus a_1 b_3 \oplus a_3 b_1.
 \end{aligned} \tag{10}$$

Base on Eq. 9, $1 \oplus a_0 \oplus a_3 \oplus a_1 a_3$ and $1 \oplus b_0 \oplus b_3 \oplus b_1 b_3$ are replaced by A_0 and B_0 . We use the same derivation to obtain $s_{m_inv}^i$:

$$\left\{ \begin{aligned}
 s_{m_inv}^3 &= A_3 \oplus B_3 \oplus a_0 b_1 \oplus a_1 b_0 \oplus a_0 a_2 b_1 \oplus a_0 a_1 b_2 \\
 &\quad \oplus a_1 a_2 b_0 \oplus b_0 b_2 a_1 \oplus b_0 b_1 a_2 \oplus b_1 b_2 a_0 \oplus a_0 a_2 b_3 \\
 &\quad \oplus a_0 a_3 b_2 \oplus a_2 a_3 b_0 \oplus b_0 b_2 a_3 \oplus b_0 b_3 a_2 \oplus b_2 b_3 a_0 \\
 s_{m_inv}^2 &= A_2 \oplus B_2 \oplus 1 \oplus a_0 b_1 \oplus a_1 b_0 \oplus a_0 b_2 \oplus a_2 b_0 \oplus \\
 &\quad a_0 b_3 \oplus a_3 b_0 \oplus a_1 b_2 \oplus a_2 b_1 \oplus a_1 b_3 \oplus a_3 b_1 \\
 &\quad \oplus a_0 a_2 b_1 \oplus a_0 a_1 b_2 \oplus a_1 a_2 b_0 \oplus b_0 b_2 a_1 \\
 &\quad \oplus b_0 b_1 a_2 \oplus b_1 b_2 a_0 \oplus a_0 a_3 b_1 \oplus a_0 a_1 b_3 \\
 &\quad \oplus a_1 a_3 b_0 \oplus b_0 b_3 a_1 \oplus b_0 b_1 a_3 \oplus b_1 b_3 b_0 \\
 &\quad \oplus a_0 a_2 b_3 \oplus a_0 a_3 b_2 \oplus a_2 a_3 b_0 \oplus b_0 b_2 a_3 \\
 &\quad \oplus b_0 b_3 a_2 \oplus b_2 b_3 a_0 \\
 s_{m_inv}^1 &= A_1 \oplus B_1 \oplus a_0 b_2 \oplus a_2 b_0 \oplus a_1 b_3 \\
 &\quad \oplus a_3 b_1 \oplus a_2 b_3 \oplus a_3 b_2 \oplus a_0 a_2 b_1 \\
 &\quad \oplus a_0 a_1 b_2 \oplus a_1 a_2 b_0 \oplus b_0 b_2 a_1 \oplus b_0 b_1 a_2 \\
 &\quad \oplus b_1 b_2 a_0 \oplus a_0 a_3 b_1 \oplus a_0 a_1 b_3 \oplus a_1 a_3 b_0 \\
 &\quad \oplus b_0 b_3 a_1 \oplus b_0 b_1 a_3 \oplus b_1 b_3 a_0 \oplus a_0 a_2 b_3 \\
 &\quad \oplus a_0 a_3 b_2 \oplus a_2 a_3 b_0 \oplus b_0 b_2 a_3 \oplus b_0 b_3 a_2 \\
 &\quad \oplus b_2 b_3 a_0 \\
 s_{m_inv}^0 &= A_0 \oplus B_0 \oplus 1 \oplus a_1 b_3 \oplus a_3 b_1.
 \end{aligned} \right. \tag{11}$$

We enumerate the 4 bits of $a_1 b_3 a_3 b_1$, where 0000, 0001, 0010, 0100, 0101, 0110, 1000, 1001, 1010, 1111 make $a_1 b_3 \oplus a_3 b_1 = 0$, and 0011, 0111, 1011, 1100, 1101, 1110 make $a_1 b_3 \oplus a_3 b_1 = 1$. Then there is a probability of 0.625 that $a_1 b_3 \oplus a_3 b_1 = 0$, so $s_{m_inv}^0$ has a probability of 0.625 to be the opposite value of $A_0 \oplus B_0$. Using the same method, the final values of $A_3 \oplus B_3$, $A_2 \oplus B_2$, $A_1 \oplus B_1$ and $A_0 \oplus B_0$ are obtained from $s_{m_inv}^3, s_{m_inv}^2, s_{m_inv}^1$ and $s_{m_inv}^0$ with probabilities of 0.625, 0.53125, 0.625 and 0.625, respectively. These probabilities are greater than 1/2 and cannot be neglected. $\square$

Proposition 3 Compared with Zahednejad's data format, more XOR information for the previous round is captured by the data format in this paper.

Proof It is well-known that a secure cipher has strong randomness. PRESENT is a block cipher designed based on Shannon's (1948) principles of diffusion, confusion, and product iteration. As a result, it possesses strong randomness. In order to facilitate a comparison of the differences between our data format and Zahednejad's data format, we impose a security assumption that each round of encryption for PRESENT exhibits strong randomness. In other words, the probability of the output value being equal to the input value is 0.5.

Take PRESENT as an example and apply XOR to 7 rounds of ciphertexts in Zahednejad's data format:

$$C_0^7 \oplus C_1^7 \oplus C_2^7 \oplus C_3^7 \oplus \dots \oplus C_{15}^7. \tag{12}$$

Eq. 12 is equal to:

$$(C_0^7 \oplus C_1^7) \oplus (C_2^7 \oplus C_3^7) \oplus \dots \oplus (C_0^7 \oplus C_{15}^7). \tag{13}$$

Meanwhile, C_j^7 obtains information of C_j^6 with a probability of 0.5, so each $C_0^7 \oplus C_j^7$ in Eq. 13 obtains information about $C_0^6 \oplus C_j^6$ with a probability of 0.5. The calculation is as follows:

$$\begin{aligned}
 p(C_0^7 \oplus C_j^7 = C_0^6 \oplus C_j^6) \\
 &= p(C_0^7 = C_0^6, C_j^7 = C_j^6) + p(C_0^7 \neq C_0^6, C_j^7 \neq C_j^6) \\
 &= 0.25 + 0.25 \\
 &= 0.5.
 \end{aligned}$$

Whereas the data format $(invS_0^7, \dots, invS_{15}^7)$ for the 7 rounds in this paper:

$$\begin{aligned}
 &S^{-1}(P^{-1}(C_0^7 \oplus C_0^7)), S^{-1}(P^{-1}(C_0^7 \oplus C_1^7)) \\
 &\dots, \\
 &S^{-1}(P^{-1}(C_0^7 \oplus C_{14}^7)), S^{-1}(P^{-1}(C_0^7 \oplus C_{15}^7)).
 \end{aligned}$$

$invS_j^7$ is defined by C_0^7 and the ciphertexts of the 6 rounds, eliminating P^{-1} , P and C_0^7 yields the following result:

$$\begin{aligned}
 &S^{-1}(P^{-1}(P(S(C_0^6)) \oplus C_0^7 \oplus P(S(C_j^6)) \oplus C_0^7)) \\
 &= S^{-1}(P^{-1}(P(S(C_0^6)) \oplus P(S(C_j^6)))) \\
 &= S^{-1}(S(C_0^6) \oplus S(C_j^6)).
 \end{aligned}$$

Apply XOR to all $invS_j^7$:

$$\begin{aligned}
 &S^{-1}(S(C_0^6) \oplus S(C_0^6)) \oplus S^{-1}(S(C_0^6) \oplus S(C_1^6)) \\
 &\oplus \dots \oplus \\
 &S^{-1}(S(C_0^6) \oplus S(C_{14}^6)) \oplus S^{-1}(S(C_0^6) \oplus S(C_{15}^6)).
 \end{aligned} \tag{14}$$

Proposition 2 demonstrates that the probability of $S^{-1}(S(C_0^6) \oplus S(C_j^6))$ in Eq. 14 obtaining information about $C_0^6 \oplus C_j^6$ is higher than 1/2 and cannot be neglected. In other words, the probability of $S^{-1}(S(C_0^6) \oplus S(C_j^6))$ in Eq. 14 obtaining information about $C_0^6 \oplus C_j^6$ is 0.625 or 0.53125, where there are more data with a probability of 0.625, whereas the probability of Eq. 13 is close to 0.5. Therefore, Eq. 14 yields more information about $C_0^6 \oplus C_j^6$. $\square$

The above analysis clearly shows that the data format in this paper will have a higher opportunity to learn the XOR information of the previous round than Zahednejad's data format. Thus, the number of attack rounds can be increased at least one round using the data format in this paper. The experiments in “Comparison of different data formats” section further support this point.

Designing the network structure

Our network is built upon DenseNet and incorporates MBConv to modify the convolutional layers of DenseNet.

Compared to other modules, MBConv can reduce the convolution parameters with the same input parameters, which allows us to create deeper network models with the same amount of computation. Simultaneously, MBConv connects the input and output of the convolutional block through skip connections, which gives the network the opportunity to access the unmodified state in the convolutional block. Therefore, we modified the convolutional layers of DenseNet using the MBConv. This alteration is expected to decrease training time and attain a level of accuracy that is unattainable by other models.

The network is composed of four components: an input layer containing the multisets, an initial convolutional layer composed of a single convolution, a Dense layer modified by the MBConv, and finally, Multiple Fully Connected Layers make up the prediction header. Figure 6 shows the different layers.

Input layer We use the Reshape operation in TensorFlow to transform the initial data from Eq. 6 into a three-dimensional $[w, \frac{2LM}{sw}, s]$. $[w, \frac{2LM}{sw}]$ represents the

characteristic information of each two-dimensional convolution channel operation, where s is the number of channels, w is the width of the channel, $\frac{2LM}{sw}$ is the height of the channel, L is the size of the cipher, and M is the size of the multiset, and $s = 8$, $L = 64$, $w = 16$, $M = 16$ for PRESENT.

Initial convolutional layer The data inflow from the input layer is expanded into N_f channels using convolution with a kernel size of 1. Then, Batch normalization and rectifier nonlinearities are applied to the output of these convolutions. Finally, the result is passed to the Dense layer.

Dense layer Each convolutional block contains the MBConv module. In the MBConv module, N_f is expanded to $2N_f$ using a 1×1 convolutional layer. Subsequently, a 3×3 depth separable convolutional layer is executed, and a DropOut operation is employed to prevent overfitting. Further to this, a 1×1 convolutional layer adjusts the number of channels to the specified size, and the inputs of the MBConv are summed with the outputs. Finally, a 1×1 convolutional layer is applied, and the inputs of the Dense layer are connected to the final outputs.

Prediction header The prediction header comprises a Flatten layer, three Fully Connected Layers (FC), and a single output unit. Flatten operations are applied to spread out the inflow of data. Then the three Fully Connected Layers are utilized to process the data from the Flatten layer. Batch normalization and rectifier nonlinearities are performed on the output of the Fully Connected Layers. The final layer utilizes the Sigmoid activation function to output a single output unit. Figure 7 shows the details of the network structure.

Network training scheme

In this paper, a 2^{21} sample training dataset and a 2^{17} sample validation dataset are used. In order to test the accuracy, a test dataset with 2^{17} samples is generated. The datasets are processed in batches of 2000. In order to optimise the results, we use a mean square error loss function with L2 weight regularization (regularization parameter $reg_param = 10^{-5}$) and the Adam algorithm. During the training process, this paper employs a decreasing learning rate schedule. The update algorithm for the decreasing learning rate is given by the Algorithm 1.

The training was conducted using Python 3.9 and Tensorflow 2.12.0 on Ubuntu 20.04 OS. Our device is a server with an Intel(R) Xeon(R) Platinum 8255C CPU at 2.50 GHz, 80GB of RAM, and two RTX A4500 20GB GPUs. Each model was repeated 10 times with each training session taking 5 h approximately. Ultimately, the model with the highest accuracy was selected as the final distinguisher.

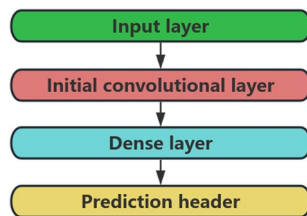


Fig. 6 Neural network architecture

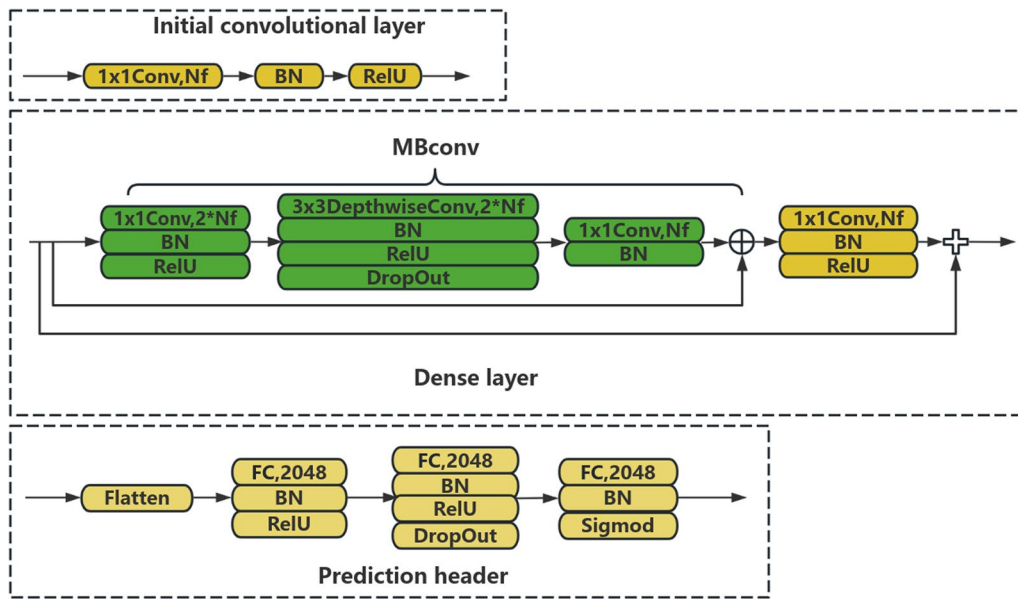


Fig. 7 Neural network structure

Algorithm 1 Learning rate decline algorithm

Require: Training batch of the model n , Current training batch i , Range of variation in learning rates (min, max)

Ensure: Learning rate for round i L_i

- 1: **if** $i \leq \frac{n}{2}$ **then**
- 2: $L_i \leftarrow max - min * i$
- 3: **else**
- 4: $L_i \leftarrow min$
- 5: **end if**
- 6: **return** L_i

Experimental results

Experiments on PRESENT

The encrypted data of PRESENT is processed by the method proposed in “Data format” section, and the number of multisets in M is 2. The processed data is fed into the neural network proposed by “Designing the network structure” section for training, and the data used for neural network training, validation and testing are randomly generated with sample sizes of 2^{21} , 2^{17} and 2^{17} , respectively.

In Fig. 8, the line chart shows the change in accuracy and loss rate for rounds 6–8. The horizontal axis represents the training batch, and the vertical axis represents the loss rate and accuracy of the neural network. From

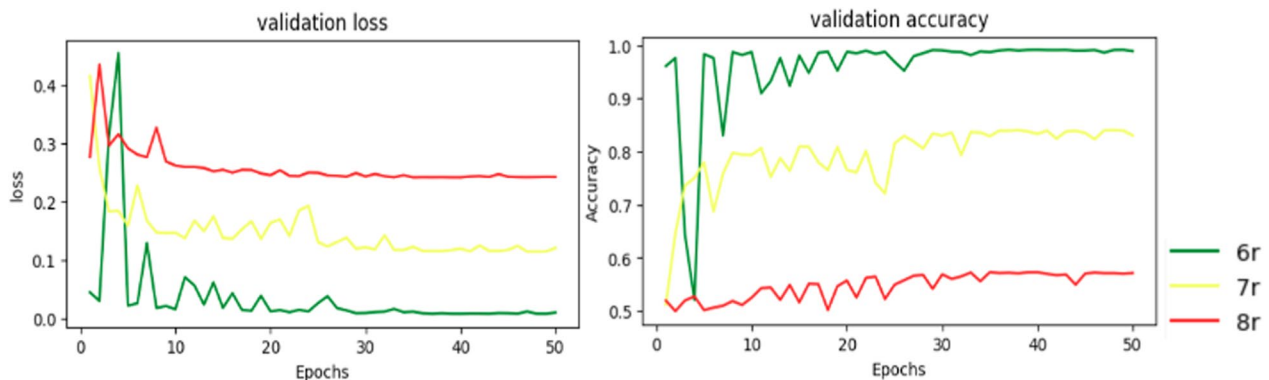


Fig. 8 PRESENT’s loss and accuracy

Table 1 Comparison of different data formats

PRESENT's round	Zahednejad's validation set accuracy (%)	Validation set accuracy of Eq. 6 (%)
5	99.59	99.53
6	95.59	98.16
7	63.13	77.31
8	–	53.69

Fig. 8, it is easy to notice that for 6 rounds the accuracy is 99.13% and the loss rate is 0.84%, for 7–8 rounds the accuracy is 82.03%, 57.32% and the loss rate is 11.51% and 24.35%. Increasing the accuracy of 8 rounds to 57.32% has differentiated the process from random guess, which is unattainable by Zahednejad's distinguisher.

Comparison of different data formats

Zahednejad used ciphertext (Eq. 4) as the input data format. The data format (Eq. 6) of this paper is given in “Data format” section.

In order to demonstrate the superiority of the data format in this paper, a comparison experiment is designed which fixes all other parameters and only changes the data format. The neural network is trained using the model in “Designing the network structure” section.

Table 1 demonstrates that our data format (the number of multisets in M is 1) achieved an accuracy of 98.16% in 6 rounds. For the 7-round integral neural distinguisher, the accuracy of the data format proposed in this paper surpasses Zahednejad's data format by 14.18%. And to our surprise, it extends the number of identification rounds to 8 rounds when $invS_0^n, invS_1^n, \dots, invS_{15}^n$ is included in the data format.

Comparison of different neural network

Zahednejad trained the distinguishers of three different neural networks (ResNet, ResNeXt, DenseNet) and obtained similar results in terms of accuracy for the three networks.

By only changing the neural network model and using Eq. 6 as the data format (the number of multisets in M is 1). In order to ensure the fairness of the experiment, the same input data is used by different networks and the input data is generated randomly. A comparison is made between the neural network in this paper and that of Zahednejad.

In Table 2, the accuracy of our network for rounds 6, 7, and 8 are reported as 98.16%, 77.31%, and 53.23%, respectively. Among the four networks, our network improved accuracy by 1.90%, 4.40% and 1.31% in rounds 6–8, respectively. This indicates that the modifications in our network contribute to the effective identification of features. Due to the channel expansion in the MBConv module within our network for extracting additional features, the training time is longer compared to ResNet. Nevertheless, it still remains lower than the training times of DenseNet and ResNeXt. Therefore, when prioritizing accuracy, our neural network proves to be effective.

Key recovery attack on SmallPresent-(8)

Due to the limitations of the device, in 2011, Blondeau and Gérard (2011) employed SmallPresent-(size) instead of PRESENT to analyze the complexity of data and time in multiple differential cryptanalysis. In 2022, Kimura (2022) also employed SmallPresent-(size) in a black-box environment to conduct a deep learning-based output prediction attack. Similarly, the devices are limited to

Algorithm 2 Key recovery attack on SmallPresent-(8).

Require: SmallPresent-(8) cipher $\mathbf{O}$, Number of random plaintexts $\mathbf{r}$, an integral neural distinguisher **Network**, chosen-plaintext generation method **Process**

Ensure: Descending Sort List of Candidate Subkeys L_{ck}

- 1: $(P_0, P_1, P_2, \dots, P_r) \leftarrow \text{RandomSelect}(\mathbf{r})$
- 2: $((M_{0,0}, \dots, M_{0,15}), \dots, (M_{r,0}, \dots, M_{r,15})) \leftarrow \text{Process}(P_0, P_1, \dots, P_r)$
- 3: $((C_{0,0}, \dots, C_{0,15}), \dots, (C_{r,0}, \dots, C_{r,15})) \leftarrow \mathbf{O}((M_{0,0}, \dots, M_{0,15}), \dots, (M_{r,0}, \dots, M_{r,15}))$
- 4: $L_{ck} \leftarrow \{\cdot\}$
- 5: **for** k in Candidate_subkeys **do**
- 6: **for** $i = 0$ to $n - 1$ **do**
- 7: $x_i^k \leftarrow \text{Network}(\text{Decrypt}(((C_{0,0}, \dots, C_{0,15}), \dots, (C_{r,0}, \dots, C_{r,15})), k))$
- 8: **end for**
- 9: $x^k \leftarrow \sum_{i=0}^{n-1} \log_2(x_i^k / (1 - x_i^k))$
- 10: Append (k, x^k) to L_{ck}
- 11: **end for**
- 12: **return** L_{ck}

Table 2 Comparison of different neural network

Neural network	Results of 6-round		Results of 7-round		Results of 8-round	
	Accuracy (%)	Time (s)	Accuracy (%)	Time (s)	Accuracy (%)	Time (s)
ResNet	96.26	2787.49	72.91	2792.69	52.36	2802.45
ResNeXt	96.32	9668.35	73.25	9555.72	52.13	9655.31
DenseNet	96.31	8080.89	73.04	8072.19	51.92	8077.53
Our network	98.16	6645.56	77.31	6698.56	53.23	6735.56

perform key recovery on PRESENT. Consequently, we refer to their work for conducting key recovery attacks on SmallPresent-(8). We believe that the key recovery attacks on the 32-bit SmallPresent-(8) can be extended to higher bits.

The active bits selection of SmallPresent-(8) is the same as that of the 5-round PRESENT integral distinguisher which is used to simplify the attack. The integral neural distinguishers for SmallPresent-(8) are trained for 5–7 rounds with accuracies of 99.32%, 72.13%, and 55.62%, respectively.

During the key recovery attacks, the ciphertexts are decrypted by all possible candidate subkeys. Then, the decrypted results are fed into integral neural distinguishers. The outputs of the integral neural distinguishers are used as the scores of the candidate subkeys. The top-ranked candidate subkey is regarded as the correct subkey suggested by the distinguisher. The specific operation procedure is shown in Algorithm 2.

- (1) Generate r plaintexts randomly. Each plaintext is expanded to a multiset based on active bits. Encrypt these multiset to obtain the corresponding ciphertext multisets.
- (2) Decrypt the ciphertext multisets with candidate subkeys.

- (3) The score x_i^k for each candidate subkey is obtained using the integral neural distinguisher.
- (4) For each subkey k , the scores x_i^k are merged into the score x^k , x^k is the score of k and sorts k in descending order.

$$x^k = \sum_{i=0}^{n-1} \log_2(x_i^k / (1 - x_i^k)).$$

In this paper, we perform key recovery attacks on the integral neural distinguishers of SmallPresent-(8). Each integral neural distinguisher repeats the key recovery attack 50 times with different keys and each attack uses 8 random plaintexts. Among these attacks, the first suggested subkey of the 5-round integral distinguisher is the correct subkey 45 times. Consequently, the 5-round integral distinguisher achieved a key recovery success rate of 90%. Key recovery success rates for 6-round and 7-round are 62% and 8%. The rank of the candidate subkeys is shown in Table 3. According to Table 3, it is not difficult to see that as the accuracy of the distinguisher decreases, it becomes more and more difficult to recover the subkey. Even the 7-round distinguisher only had 4 successful attacks.

Table 3 The rank of the candidate subkeys

Ranking of the Correct Subkeys		1	2	3–5	5–10	10–20	Others
Number of attacks	$5r$	45	2	2	1	0	0
	$6r$	31	8	7	3	0	1
	$7r$	4	1	4	4	5	32

Table 4 Total number of incorrect guesses for subkey bits

Total number of incorrect guesses		0	1	2	3	4	Others
Number of attacks	$5r$	45	3	1	1	0	0
	$6r$	31	9	5	0	3	2
	$7r$	4	1	4	3	2	36

Table 5 Percentage of correct guesses for subkey bits

Subkey bits	1 (%)	2 (%)	3 (%)	4 (%)	5 (%)	6 (%)	7 (%)	8 (%)	9 (%)	10 (%)	11 (%)	12 (%)
Percentage of correct guesses	5r	100	100	100	100	100	100	100	100	100	100	100
	6r	100	100	100	100	100	100	100	100	100	100	100
	7r	100	100	100	100	100	100	100	100	100	98	98
Subkey bits	13 (%)	14 (%)	15 (%)	16 (%)	17 (%)	18 (%)	19 (%)	20 (%)	21 (%)	22 (%)	23 (%)	24 (%)
Percentage of correct guesses	5r	100	100	100	98	100	96	100	100	100	96	96
	6r	100	100	100	98	98	92	96	94	96	92	96
	7r	96	92	90	81	66	68	60	56	60	64	64
Subkey bits	25 (%)	26 (%)	27 (%)	28 (%)	29 (%)	30 (%)	31 (%)	32 (%)				
Percentage of correct guesses	5r	94	94	100	100	100	100	100				
	6r	90	98	88	96	94	98	86				
	7r	58	66	50	60	56	58	68				

Additionally, our aim is to ensure correctness for each bit during key recovery attacks. In this paper, a successful guess is defined as the scenario where the subkey guessed by the neural distinguisher differs from the correct subkey within two bits. Then the incorrect bits can be eliminated by using the exhaustive method. The results of the experiments can be found in Tables 4 and 5.

The success rate of the 5-round neural distinguisher for SmallPresent-(8) in Table 4 is 98%, with 45 attacks having completely accurate subkey guesses. Among the experiments with incorrect guesses, 3 attacks have only 1 bit errors, and 1 attack have 2 bits guessed incorrectly. Only 1 attack fail to meet the criteria established, and none of them have more than 3 bits of incorrect guesses. Simultaneously, the success rate of the 6-round neural distinguisher for SmallPresent-(8) is 90%, which has 5 attacks with more than 2 bits of error bits. However, the key recovery accuracy of the 7-round distinguisher drops to 18% and there are 36 attacks with more than 4 incorrect bits.

Table 5 statistics the success rate of recovery for each bit. Although the 5-round neural distinguisher incorrectly identifies the key bits, all of these key bits have a success rate of over 90%. For the 6-round distinguisher, only bits 27 and 31 have success rates of less than 90%. But it is still very successful to get such a high bit recovery success rate using the classical 5-round integral distinguisher. Surprisingly, although the neural distinguishers are different, the success rate of the top 10 bits are all 100%.

Conclusions

This paper improves the integral neural distinguisher of PRESENT and performs key recovery attacks on SmallPresent-(8). Firstly, this paper obtains more derived features by adopting the data format $(invP_0^n, invP_1^n, \dots, invP_{15}^n, invS_0^n, invS_1^n, \dots, invS_{15}^n)$. Additionally, we implement the ideas from the MBConv module to enhance the convolutional layers of DenseNet. As a result, this paper improves the accuracy of the 6-round and 7-round PRESENT integral neural distinguisher to 99.13% and 82.03%, respectively. And this improvement result in an accuracy of 57.32% for PRESENT's 8-round integral neural distinguisher, which is unattainable by Zahednejad's distinguisher. In order to confirm the effectiveness of the modifications to the neural network, we perform experiments with 4 different neural networks, and the results show that there are some advantages of our network structure in pursuit of accuracy. Finally, we perform key recovery attacks on SmallPresent-(8). The results demonstrate that the key recovery success rates for 5-round and 6-round are 98% and 90%, considering error bits within a range of two bits.

In this paper, the modifications to the data format led to an improvement in accuracy. It shows that the derived features exposed by the data format have a significant effect on the neural network. Simultaneously, we observe subtle differences in the results among different neural networks. Therefore, in the future, we will explore the enhancement possibilities by combining Zahednejad's data format with the data format of this paper and try to find a better data format. Simultaneously, we will attempt to identify the reasons for variations in neural network results, aiming to design an integral neural distinguisher with higher accuracy. We will also attempt to extend the methods to other block ciphers.

Acknowledgements

Not applicable.

Author contributions

Wanqing Wu: Conceptualization, methodology, validation, writing review and editing. Mingyu Guo: Investigation, formal analysis, writing review and editing. All authors read and approved the final manuscript.

Funding

Not applicable.

Availability of data and materials

With the exception of the data given in this paper, the code and other data can be accessed at the following link: <https://github.com/Gmy1232/Integral-Neural-Distinguisher-for-PRESENT>.

Declarations

Competing interests

The authors declare that they have no competing interests.

Received: 4 January 2024 Accepted: 22 May 2024

Published online: 10 December 2024

References

- Biryukov A, Shamir A (2001) Structural cryptanalysis of SASAS. Springer, pp 395–405
- Blondeau C, Gérard B (2011) Multiple differential cryptanalysis: theory and practice. Springer, pp 35–54
- Bogdanov A et al (2007) Present: an ultra-lightweight block cipher. Springer, pp 450–466
- Chen Y, Yu H (2021) A new neural distinguisher model considering derived features from multiple ciphertext pairs. IACR Cryptol ePrint Arch 2021:310
- Cho JY (2010) Linear cryptanalysis of reduced-round present. Springer, pp 302–317
- Chollet F (2017) Xception: deep learning with depthwise separable convolutions, pp 1251–1258
- Collard B, Standaert F-X (2009) A statistical saturation attack against the block cipher present. Springer, pp 195–210
- Daemen J, Knudsen L, Rijmen V (1997) The block cipher square. Springer, pp 149–165
- Gohr A (2019) Improving attacks on round-reduced speck32/64 using deep learning. Springer, pp 150–179
- He K, Zhang X, Ren S, Sun J (2016) Deep residual learning for image recognition, pp 770–778
- Hou B, Li Y, Zhao H, Wu B (2020) Linear attack on round-reduced des using deep learning. Springer, pp 131–145
- Howard A et al (2019) Searching for mobilenetv3, pp 1314–1324

- Huang G, Liu Z, Van Der Maaten L, Weinberger K Q (2017) Densely connected convolutional networks, pp 4700–4708
- Jain A, Kohli V, Mishra G (2020) Deep learning based differential distinguisher for lightweight cipher present. *Cryptology ePrint Archive*
- Kimura H et al (2022) Output prediction attacks on block ciphers using deep learning. Springer, pp 248–276
- Knudsen L, Wagner D (2002) Integral cryptanalysis. Springer, pp 112–127
- Leander G (2010) Small scale variants of the block cipher present. *Cryptology ePrint Archive*
- Lucks S (2001) The saturation attack—a bait for twofish. Springer, pp 1–15
- Nakahara J, Sepehrdad P, Zhang B, Wang M (2009) Linear (hull) and algebraic cryptanalysis of the block cipher present. Springer, pp 58–75
- Shannon CE (1948) A mathematical theory of communication. *Bell Syst Tech J* 27:379–423
- Tan M, Le Q (2019) Efficientnet: rethinking model scaling for convolutional neural networks, pp 6105–6114 (PMLR)
- Tan M et al (2019) Mnasnet: platform-aware neural architecture search for mobile, pp 2820–2828
- Todo Y (2015) Structural evaluation by generalized integral property. Springer, pp 287–314
- Todo Y, Morii M (2016) Compact representation for division property. Springer, pp 19–35
- Wang M (2008) Differential cryptanalysis of reduced-round present. Springer, pp 40–49
- Wang S, Xiang Z, Zeng X, Zhang S (2018) Improved integral attacks on present-80. Springer, pp 163–182
- Wu S, Wang M (2013) Integral attacks on reduced-round present. Springer, pp 331–345
- Xiang Z, Zhang W, Bao Z, Lin D (2016) Applying MILP method to searching integral distinguishers based on division property for 6 lightweight block ciphers. Springer, pp 648–678
- Xie S, Girshick R, Dollár P, Tu Z, He K (2017) Aggregated residual transformations for deep neural networks, pp 1492–1500
- Yang L, Wang M, Qiao S (2009) Side channel cube attack on present. Springer, pp 379–391
- Z'aba MR, Raddum H, Henriksen M, Dawson E (2008) Bit-pattern based integral attack. Springer, pp 363–381
- Zahednejad B, Lyu L (2022) An improved integral distinguisher scheme based on neural networks. *Int J Intell Syst* 37:7584–7613

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.